迷你搜索引擎项目

测试报告 02：正确性与独立对拍

黄金样例、Python 参考实现与全量 brute-force

版本 1.0　测试日期 2026-10-19

报告编号	TR-02
被测对象	迷你搜索引擎（C++17，`mse`）
被测版本	分支 `test/reports`，基于 `origin/main` 加注释与测试改动
测试环境	WSL2（Linux 6.18），g++ 15.2，CMake 3.31.6，Python 3
测试日期	2026-10-19
总体结论	黄金样例、词频 CSV、全词表 brute-force、短语与磁盘词典均一致

1　目标与范围

本报告回答一个与单元测试不同的问题：**C++ 实现是否忠实于规格**。单元测试只能证明“代码符合我理解的规格”；如果规格本身被误解，测试也会一起错。因此这里使用三类独立证据：

1. **黄金样例**：讲义给出的四文档例子，人工逐项核对 posting 表；
2. **独立参考实现**：Python 重新实现分词、停用词、Porter 词干化与索引解析，从原始文本重建期望值；
3. **全量 brute-force**：不只抽查几个词，而是对全部 14,937 个词干逐个比较 df、tf、docID 与 positions。

阈值实验与规模测试分别见 TR-03、TR-04；本报告不重复。

2　方法

2.1　黄金样例

`tests/fixtures/golden4/` 复刻讲义 1.2 节的四篇文档：

表 1: 黄金样例文档

docID	文本
1	Gold silver truck
2	Shipment of gold damaged in a fire
3	Delivery of silver arrived in a silver truck
4	Shipment of gold arrived in a truck

原始构建（无停用词、无词干化）得到 4 个文档、11 个词、25 个 token。程序输出的 posting 表与讲义逐项一致：

表 2: 黄金样例 posting 表

词	posting（docID, tf）
a	(2,1) (3,1) (4,1)
arrived	(3,1) (4,1)
damaged	(2,1)
delivery	(3,1)
fire	(2,1)
gold	(1,1) (2,1) (4,1)
in	(2,1) (3,1) (4,1)
of	(2,1) (3,1) (4,1)
shipment	(2,1) (4,1)
silver	(1,1) (3,2)
truck	(1,1) (3,1) (4,1)

位置校验：doc1 中 `silver` 在 position 1；doc3 中 `silver` 在 positions 2 和 6。

一处参考材料复核：讲义表格把 a 记为 (2,1)(4,1)，但 doc3 的原文是 “Delivery of silver arrived in **a** silver truck”，按句子重算应为 (2,1)(3,1)(4,1)。程序输出与原文一致。测试因此以原始句子为准，并在用例中显式记录这处差异。这说明黄金样例本身也需要复核，不能盲目相信二手表格。

停用词版本（过滤 `a/of/in`）得到 8 个词；`silver` 在 doc3 中的位置重新编号为 1 和 3，验证了 “停用词不占 position” 的语义。

2.2 独立 Python 实现

三个脚本各自独立实现规格，不调用 C++ 的中间结果：

- `tools/check_wordcount.py`：独立分词、停用词过滤与 Porter 词干化，生成与 `mse count` 同格式的 CSV；
- `tools/brute_force.py`：从 `data/text/` 重新扫描语料，重建完整倒排索引；同时用独立二进制解析器读取 `index.bin`（v1 或 v2），逐词比较；
- `tools/check_phrases.py`：从语料随机抽取短语，在 Python 的规范化 token 流上做暴力相邻匹配，再与 `mse query --phrase` 的文档集比较。

“独立”指实现路径独立：Python 侧不链接 C++ 代码，也不读取 C++ 的中间结构；只有原始 manifest 与文本层是共同输入。

3 对拍结果总表

表 3: 独立对拍结果

对拍项	规模	结果
逐文档 token 数（C++ vs Python）	195 文档	100% 一致，866,626 token
`wordcount.csv` 默认配置	25,705 行	逐字节一致
`wordcount.csv --keep-speakers`	25,705 行	逐字节一致
`wordcount.csv --stop-words`	556,866 kept / 309,760 removed	逐字节一致
`wordcount.csv --stem`	14,947 词干	逐字节一致（NLTK `MARTIN_EXTENSIONS`）
`wordcount.csv --stem --stop-words`	14,937 词干	逐字节一致
词典随机工作负载	20,000 次操作 / 300 词	splay 与 hash 词表完全一致
索引文件（splay vs hash）	3,685,655 B	逐字节一致
索引文件（splay vs `std::map`）	3,685,655 B	逐字节一致
`brute_force.py` 全词表（v1）	14,937 词干	df、tf、docID、positions 全项一致
`brute_force.py` 全词表（v2）	14,937 词干	独立解析 v2，全项一致
`brute_force.py` raw 版	25,705 词	全项一致
块式构建 vs 内存构建	17 个 run / 4,166,556 B 临时	索引逐字节一致
磁盘 B+Tree `dump --verify-all`	14,937 词	0 差异
磁盘词典查询 vs 全量加载	`osric` 单词查询	文本与 JSON 输出逐字节一致
短语查询 vs 独立扫描（v1）	303 条随机短语	返回文档集 100% 一致
短语查询 vs 独立扫描（v2）	303 条随机短语	返回文档集 100% 一致
RFC 语料全链路	3 文档 / 2,660 词干	count CSV 逐字节一致；brute-force 全项一致

4 关键对拍细节

4.1 词频 CSV：逐字节一致

`mse count` 与 `check_wordcount.py` 在五种配置下生成的文件用 `diff` 比较，均无差异。词频 CSV 同时包含 rank、tf、df、df_ratio、tf_share 与熵特征，因此这项比较不仅验证“词和次数”，也验证了熵公式、排序与浮点格式。

4.2 全词表 brute-force

`brute_force.py` 不抽样：它对索引中的每一个 term 重建 posting，再逐项比较。正式配置下的输出为：

```
index    : build/index.bin
documents: 195   terms: 14937   tokens: 556866
compared : 14937 terms (df, tf, docIDs and positions for every posting)
result   : OK -- independent scan and index file agree
```

这也覆盖了停用词位置语义：停用词不占 position，因此短语相邻性在过滤后的 token 流上仍然成立。

4.3 短语对拍

`check_phrases.py` 从语料随机抽取短语（默认 300 条，实际采样 305 条）：

```
corpus   : data/manifest.tsv (195 documents)
index    : build/index.bin
phrases  : 305 sampled, 303 checked, 2 skipped
result   : OK -- every checked phrase matches the brute-force scan
```

两条被跳过：一条在规范化后只剩一个可检索词（退化为单词查询），另一条在索引中没有任何可检索词（全部是停用词）。跳过原因会打印，不是静默忽略。同一脚本对 v1 与 v2 各跑一遍，结果一致。

4.4 三种词典产出同一索引

splay、hash、`std::map` 三种词典分别构建正式索引，三个文件逐字节相同（3,685,655 B）。随机工作负载测试（20,000 次操作、300 个词）也比较了 splay 与 hash 的词表，结果完全一致。字典实现只影响构建期性能，不影响冻结后的索引内容。

4.5 v2 压缩索引的独立解析

v2 把 postings 改为 docID gap、tf、position delta，并用字节对齐 varint 编码。`brute_force.py` 内置独立 v2 解析器，不调用 C++ 解码函数；14,937 个词全部一致。另有 varint 边界测试覆盖 6/14/22/30 位的上下界与截断拒绝。

4.6 磁盘 B+Tree 词典

正式索引的 sidecar 为 237,568 B（58 页 × 4 KiB，树高 2）。`dump --verify-all` 逐个 term 经 B+Tree 定位并解析 posting，结果：

```
verified   : 14937 terms, 0 mismatches
dict reads : 114 page reads, 29819 cache hits
```

同一索引的全量加载路径与磁盘路径对 `osric` 的文本输出、JSON 输出逐字节一致。

4.7 第二语料：RFC

`data2/` 收录 RFC 1035/5321/7231 三篇 plain 文本。用同一套代码构建（动态熵停用词 58 词 + 词干化）得到 2,660 个词干、4,594 条 posting、45,370 个 token、308,164 B。`brute_force.py` 对 2,660 个词全项一致；`mse count` 的 CSV 与 Python 版本逐字节一致。这证明引擎不依赖莎士比亚语料的具体格式。

5 对拍发现并修复的问题

对拍与测试共抓出 13 个真实问题。下面保留完整清单，因为它们比“全部通过”更能说明测试的有效性。

表 4: 测试与对拍发现的问题

#	现象	原因	处理
1	对拍脚本比 C++ 少算 4,340 个 token	只按行首判断标记行，把 `[To HELENA]` 之类台词误判为标题行	改为行首与行尾同时匹配
2	正文混入 `aside`、`sings` 和人名	610 处方括号舞台提示被当作正文	提取阶段把行首 `[...]` 重定位为 `(...)` 舞台行；净减 849 token
3	过滤版词频的 `tf_share` 分母错误	C++ 曾把过滤前 token 总数记入 `tokens`	只累计通过过滤器的 token；`stopwordTokens` 单独累计
4	41 个词形被过度词干化（如 `agreem`）	误以为后缀条件失败后应继续尝试更短后缀	改为“首个匹配说了算、条件失败即停”
5	伸展树插入后段错误	插入分支保留了 `Node*&` 子指针引用，旋转后引用失效	先把新节点指针复制到局部变量，再 splay
6	索引加载误报“index file is truncated”	`Reader` 构造时传入了临时 vector，整句结束后引用悬垂	先把文件内容存入具名变量，再构造 Reader

#	现象	原因	处理
7	v2 压缩索引的 posting offset 全部偏移	计算压缩偏移时漏加了 term 表自身大小	复用与 v1 相同的 `computeTermOffsets()`
8	块式构建在真实语料上报“run 文件截断”	Windows 上 run 文件用文本模式读取，CRLF 被翻译进二进制流	显式 `std::ios::binary`
9	`MapDictionary` 的 `sortedEntries()` 产出空词表	`try_emplace` 只填 map 的键，`TermEntry::term` 未镜像	插入后手动设置 `entry.term`
10	B+Tree 的 I2 区间校验误报	分隔键是真实键的副本，右子树最左键等于左分隔键	I2 改为左闭区间 `[key[i-1], key[i])`
11	变长键下 B+Tree 的 50% 填充不变量被违反	按条数对半分裂在键长偏斜时不保证字节下界	改为“合法窗口内取字节中点”分裂
12	磁盘词典查询头部显示“1 terms”	lazy 路径只把被查询词挂到内存索引	头部改用 sidecar 的 `keyCount()`
13	30M/50M 键的 B+Tree bulk-load 报 I2 违反	内层借用时插入了被移动子树的最小键，而非原 `last` 的首键	插入 `last` 原来的首键作为边界

6 复现方式

```
# 词频 CSV 对拍
python3 tools/check_wordcount.py > build/wordcount_py.csv
diff build/wordcount.csv build/wordcount_py.csv

# 全词表 brute-force（v1）
python3 tools/brute_force.py --stop-words data/stopwords.txt --stem

# 短语对拍（v1；把 index 换成 v2 索引即可对拍压缩格式）
python3 tools/check_phrases.py --binary ./build/mse \
    --index build/index.bin --stop-words data/stopwords.txt --stem --queries 300

# 磁盘词典全词表校验
./build/mse dump --index build/index.bin --dict-file build/index.bpt --verify-all

# 需要的索引
./build/mse build --index build/index.bin --dict-file build/index.bpt \
    --stop-words data/stopwords.txt --stem
```

上述命令都已接入 CTest（`crosscheck_*`），日常回归只需 `ctest --test-dir build -R crosscheck`。

7 结论

- 黄金样例的 posting 表与原始文本一致，并纠正了参考表格中 `a` 的一处错误；
- 五种词频配置的 CSV 与独立 Python 实现逐字节一致；
- 14,937 个词干的 df、tf、docID、positions 全量对拍通过（v1 与 v2）；
- 303 条随机短语在 v1/v2 上均与独立扫描一致；
- 三种词典实现产出逐字节相同的索引；
- 磁盘 B+Tree 的 14,937 个词 0 差异；
- RFC 第二语料端到端一致，证明引擎不依赖莎士比亚格式。

结论：在当前语料与当前规格下，C++ 索引与独立 Python 参考实现达到逐字节/逐项一致；测试不是“再跑一遍 C++”，而是从原始文本重建期望值。